Pokhara University Faculty of Management Studies

Course Code: CMP 176
Course Title: Data Structure and Algorithm
Nature of the Course: Theory & Practical
Level: Bachelor

Credits:3[Hrs]
Total Lectures: 48 hours
Program: BCSIT
Year: I Semester: II

1. Course Description

This course offers a thorough examination of the core principles of data structures and algorithms, crucial for computer science students. It starts with an introduction to data structures and advances through abstract data types, algorithm analysis, and design techniques. Topics covered include recursion, stacks, queues, linked lists, trees, sorting, searching, hashing, and graph algorithms, with in-depth explanations and practical examples. Through hands-on exercises, students gain proficiency in designing, implementing, and analyzing these structures and algorithms, with a focus on understanding their efficiency and real-world applications. Upon completion, students are equipped with the essential knowledge and skills to address intricate problems and develop efficient software solutions.

2. General Objectives

- Provide a comprehensive understanding of fundamental data structures and algorithms.
- Equip students with proficiency in designing, implementing, and analyzing data structures and algorithms.
- Foster a deep comprehension of the efficiency and practical implications of different algorithms.
- Prepare students to tackle complex problems and develop effective software solutions.

3. Specific Objectives and Contents

Specific Objectives	**Content**
• Explain fundamental concepts and importance of data structures. • Define and classify different types of data structures. • Differentiate between data structures and abstract data types (ADTs). • Analyze and evaluate algorithms in terms of time and space complexity.	**Unit I: Introduction to data Structure [3 Hrs.]** **1.1 introduction** **1.2** Definition, **1.3** Classification of data structure **1.4** Abstract Data Type **1.5** Comparison between Data structure and ADT, **1.6** Algorithm **1.7** Analysis algorithm **1.8** Design algorithm ▪ Incremental approach ▪ Divide and conquer **1.9** Performance analysis and measurement

	▪ Space complexity ▪ Time complexity
• Discuss the principles and characteristics of recursion. • Apply recursion to solve problems and implement algorithms efficiently.	**Unit II: Recursion [4 Hrs.]** 2.1 Introduction to Recursion 2.2 Principle of Recursion 2.3 Types of Recursions ▪ Direct, ▪ Indirect, ▪ Linear ▪ Tail recursion 2.4 Recursion Examples ▪ TOH, ▪ Fibonacci Series, 2.5 Application of Recursion
• Become proficient in using stacks and understanding their terms. • Proficiency in implementing stack algorithms such as POP and PUSH. • Application of stacks in solving problems like reverse string, postfix expression evaluation, and infix to postfix conversion.	**Unit III: Stacks [4 Hrs.]** 3.1 Introduction 3.2 Operation stack 3.3 Stack terminology 3.4 Algorithm for POP and PUSH 3.5 Stack Applications ▪ Stack frame ▪ Reverse string ▪ Calculation of postfix expression ▪ A notation conversion 3.6 Algorithm for converting infix expression to postfix from.
• Discuss of queue terminology and operations. • Gain proficiency in implementing queue insertion and deletion algorithms. • Recognize queue variations and their applications, such as circular queues and priority queues.	**Unit IV: Queue [4 Hrs.]** 4.1 Introduction 4.2 Queue terminology 4.3 Algorithm for insertion queue 4.4 Algorithm for deletions in queue 4.5 Limitation of simple queue 4.6 Variation in a queue ▪ A circular queue ▪ Priority of queue 4.7 Application of queue
• Comprehend the advantages and disadvantages of linked lists. • Define key terms associated with linked lists, such as data field and linked field. • Demonstrate the representation of a linear linked list. • Execute operations on linked lists including creation, insertion, deletion, traversal, searching, concatenation, and display.	**Unit V: Linked List [5 Hrs.]** 5.1 Introduction 5.2 Linked List ▪ Advantage and disadvantage 5.3 Key term ▪ Data field ▪ Linked field 5.4 Representation liner linked list 5.5 Operation of linked list ▪ Creation

• Identify and differentiate between various types of linked lists, including single, double, circular, and circular double linked lists.	▪ Insertion ▪ Deletion ▪ Traversing ▪ Searching ▪ Concatenation ▪ Display 5.6 Types of linked list ▪ Single linked list ▪ Double linked list ▪ Circular linked list ▪ Circular double linked list 5.7 Create single linked list 5.8 Insertion of single linked list at specific position 5.9 Deletion of single linked list at specific position 5.10 Application: addition of two polynomials
• Define tree terminology and apply it, including concepts like strictly binary trees and complete binary trees. • Understand different representations of binary trees, like array and linked list representations. • Create a binary tree and implement various traversal methods. • Perform operations on binary search trees, including insertion, search, and deletion, and understand their properties. • Learn about AVL balanced trees, their principles, properties, and how they ensure efficient data structure operations by maintaining balance and minimizing search times .	**Unit VI: Trees [7 Hrs.]** 6.1 Introduction 6.2 Tree terminology 6.3 Binary tree ▪ Strictly binary tree ▪ Complete binary tree ▪ Extended binary tree 6.4 Binary tree representation ▪ Array representation of binary tree ▪ Linked list representation of binary tree 6.5 Create binary tree 6.6 A traversal of a binary tree ▪ Preorder traversal ▪ Inorder traversal ▪ Post order traversal 6.7 Binary search tree ▪ Insertion ▪ Search ▪ Deletion 6.8 Tree Height, Level and Depth, 6.9 Balanced Tree: AVL Balanced Trees 6.10The Huffman Algorithm 6.11 B-Tree
• Understand the distinction between internal and external sorting techniques. • Learn various common sorting algorithms such as Bubble Sort, Insertion Sort, Selection Sort,	**Unit VII: Sorting[7 Hrs.]** 7.1 Introduction 7.2 Internal & External Sort, 7.3 Common sorting algorithm

Quick Sort, Merge Sort, Shell Sort, and Binary Sort. • Analyze the efficiency of sorting algorithms and their performance using Big O notation. • Implement sorting algorithms and evaluate their effectiveness in sorting large datasets.	▪ Bubble sort ▪ Insertion ▪ Selection Sort, ▪ Quick Sort, ▪ Merge Sort, ▪ Shell Sort, ▪ Binary Sort, 7.4 Efficiency of Sorting, Big 'O' Notation
• Various searching techniques including Sequential Search, Binary Search, and Tree Search. • Learn about hashing techniques, including hash functions and hash tables, and their applications. • Explore collision resolution techniques such as linear probing, quadratic probing, and double hashing. • Compare the efficiency of different search techniques and hashing methods to determine their suitability for various applications.	**Unit VIII: Searching [5 Hrs.]** 8.1 Introduction 8.2 Searching Technique ▪ Sequential Search, ▪ Binary Search, ▪ Tree Search, 8.3 Hashing: ▪ Hash functions o Character of a good hash functions o Type of hash function ▪ Hash tables and application 8.4 Collision resolution techniques 8.5 Hashing with open addressing ▪ Linear probing ▪ Quadratic probing ▪ Double hashing 8.6 Hashing with chaining 8.7 Rehashing 8.8 Efficiency comparison of different search technique
• Understand the fundamental concepts and terminology related to graphs. • Learn about different types of graphs, including undirected and directed graphs. • Explore graph representation techniques and graph traversal algorithms such as Breadth-First Search (BFS) and Depth-First Search (DFS). • Study spanning trees, minimum spanning trees, and algorithms such as Kruskal's algorithm and Prim's algorithm for their construction.	**Unit 9: Graph [7 Hrs.]** 9.1 Introduction 9.2 Graph terminology 9.3 Types of Graphs ▪ Undirected graph ▪ Directed graph 9.4 Graph representation 9.5 Graph Traversal ▪ Breath first search ▪ Depth first search 9.6 Spanning tree and Minimum spanning tree ▪ Kruskal's algorithm ▪ Prime algorithm 9.7 Shortest Path problem ▪ Dijkstrs's Algorithm 9.8 Application graph
• Learn the concept of asymptotic notations,	**Unit 10: Growth Functions [2 Hrs.]**

including Big O, Omega, and Theta notation. • Learn the limitations of Big O notation and its applicability in analyzing algorithmic complexity.	10.1. Introduction to Asymptotic Notations 10.2 Big Oh notation 10.3 Omega notation 10.4 Theata notation 10.5 Limatitation of big Oh notation

4. Laboratory Work

It builds the foundation on how to write a program using any high-level language. Hence, this course requires a lot of programming practice so that students will be able to develop good logic building and program developing capability which is essential throughout the course.

Some important contents that should be included in lab exercises are as follows:

1. Implementations of different operations related to Stack.
2. Implementation of different operations related to linear and circular queue.
3. Solution of TOH and Fibonacci Series using Recursion.
4. Implementations of different operations related to singly linked list.
5. Implementation of Trees: AVL trees, Balancing AVL.
6. Implementation of merge sort.
7. Implementation of different searching technique: sequential, Tree and Binary.
8. Implementation of Graphs: Graph traversal
9. Implementation of Hashing

Note: Each of the above lab session should cover more than 4 hours of practical work.

5. Methods of Instruction

- Lecture
- Group discussion
- Question-answers
- Demonstration and discussion
- Presentations
- Guest lectures
- Group work/project work
- Problem solving
- Simulation
- Tutorials

6. Evaluation system and Student's Responsibilities

Evaluation System

In addition to the formal exam(s), the internal evaluation of a student may consist of quizzes, assignments, lab reports, projects, class participation, etc. The tabular presentation of the internal evaluation is as follows.

External Evaluation	**Marks**	**Internal Evaluation**	**Weight**	**Marks**
Semester-End examination	50	**Theory**		30
		Attendance & Class Participation	10%	
		Assignments	20%	
		Presentations/Quizzes	10%	

		Internal Assessment	60%	
		Practical		
		Attendance & Class Participation	10%	20
		Lab Report/Project Report	20%	
		Practical Exam/Project Work	40%	
		Viva	30%	
Total External	50	Total Internal		50
Full Marks: 50 + 50 = 100				

Student's Requirements

Each student must secure at least 45% marks separately in both internal assessment and practical evaluation with 80% attendance in the class in order to appear in the Semester End Examination. Failing to get such score will be given NOT QUALIFIED (NQ) to appear the Semester-End Examinations. Students are advised to attend all the classes, formal exam, test, etc. and complete all the assignments within the specified time period. ***Students are required to complete all the requirements defined for the completion of the courses.***

Prescribed Books and References <u>Text Books</u>

Langsam, Y., Augenstein, M. J., & Tanenbaum, A. M. (2019). Data Structures using C and C++. PHI

<u>Reference Books</u>

1 Rowe, G. W. (1997). Introduction to Data Structures and Algorithms with C and C++. PHI
2 Lafore, R. (2002). Data Structures and Algorithms in Java. Sams Publishing
3 Baluja, G. S. (2016). Data Structures through C. Dhanpat Rai & Co